# System Verification and Validation Plan for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

October 29, 2026

# Revision History

| Date | Version | Notes |
|---|---|---|
| Oct 26 | 1.0 | Added system tests for authentication and task creation |
| Date 2 | 1.1 | Notes |

# Contents


**1 Symbols, Abbreviations, and Acronyms** **iv**

**2 General Information** **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 1

**3 Plan** **1**
- 3.1 Verification and Validation Team . . . 1
- 3.2 SRS Verification . . . 1
- 3.3 Design Verification . . . 1
- 3.4 Verification and Validation Plan Verification . . . 1
- 3.5 Implementation Verification . . . 1
- 3.6 Automated Testing and Verification Tools . . . 1
- 3.7 Software Validation . . . 1

**4 System Tests** **1**
- 4.1 Tests for Functional Requirements . . . 2
  - 4.1.1 Accounts and Authentication . . . 2
  - 4.1.2 Inference Task Creation . . . 7
  - 4.1.3 Provider Registration and Staking . . . 9
  - 4.1.4 Inference Requests . . . 14
- 4.2 Tests for Nonfunctional Requirements . . . 18
  - 4.2.1 Area of Testing1 . . . 18
  - 4.2.2 Area of Testing2 . . . 19
- 4.3 Traceability Between Test Cases and Requirements . . . 19

**5 Unit Test Description** **19**
- 5.1 Unit Testing Scope . . . 19
- 5.2 Tests for Functional Requirements . . . 19
  - 5.2.1 Module 1 . . . 19
  - 5.2.2 Module 2 . . . 20
- 5.3 Tests for Nonfunctional Requirements . . . 20
  - 5.3.1 Module ? . . . 20
  - 5.3.2 Module ? . . . 21

5.4 Traceability Between Test Cases and Modules . . . . . 21

**6 Appendix** **22**
6.1 Symbolic Parameters . . . . . 22
6.2 Usability Survey Questions? . . . . . 22

**7 Appendix — Generative AI Use Disclosure** **23**
7.1 AI Tools Used . . . . . 23
7.2 How and Where Used . . . . . 23
7.3 Verification of AI-Generated Content . . . . . 23

**8 Appendix — Reflection** **24**

# List of Tables

# List of Figures

# 1 Symbols, Abbreviations, and Acronyms

| symbol | description |
|---|---|
| T | Test |

This document ...

# 2 General Information

## 2.1 Summary

## 2.2 Objectives

## 2.3 Custom Documents

## 2.4 Relevant Documentation

?

# 3 Plan

## 3.1 Verification and Validation Team

## 3.2 SRS Verification

## 3.3 Design Verification

## 3.4 Verification and Validation Plan Verification

## 3.5 Implementation Verification

## 3.6 Automated Testing and Verification Tools

## 3.7 Software Validation

# 4 System Tests

This section describes the tests that will be performed to verify that the entire integrated system meets the functional and non-functional requirements specified in the SRS. The system test cases below are be descrbed by the following characteristics:

- Control: Is this test manual or automatic?

- Initial State: What is the state of the system before the test is performed?
- Input: What is the input to the system for this test?
- Output: What is the expected output of the system for this test?
- Test Case Derivation: What FRs or NFRs does this test case cover?
- How test will be performed: What is the abstract procedure for performing the test?

## 4.1 Tests for Functional Requirements

### 4.1.1 Accounts and Authentication

These tests verify the account authentication requirements in the SRS, including authentication with a compatible Web3 wallet extension and an embedded wallet.

#### Authentication Using a Web3 Wallet Extension

1. ST-AUTH-EXT-1

   Control: Manual

   Initial State: The application is running in a supported browser on its public page. A compatible Web3 wallet extension is installed and active. Test profiles `EXT-USER-01` and `EXT-USER-02` are associated with wallet fixtures `EXT-WALLET-01` and `EXT-WALLET-02`; neither user is signed in.

   Input:

   - Connect `EXT-WALLET-01` and approve the verification-message signature requested by the application.
   - Connect `EXT-WALLET-02` and approve its verification-message signature.
   - Connect `EXT-WALLET-01` and reject the signature request.

   Output:

- For the approved signature from `EXT-WALLET-01`, the user is authenticated as `EXT-USER-01`, and that user's profile and authenticated features are available.
- For the approved signature from `EXT-WALLET-02`, the user is authenticated as `EXT-USER-02`, and that user's profile and authenticated features are available.
- The user remains unauthenticated, no profile or authenticated features become available.

Test Case Derivation: FR1 requires the authentication response to verify ownership of the connected wallet address.

How test will be performed: For each wallet, start from a signed-out session, connect the specified extension account, and approve the verification-message signature request. Confirm that `EXT-WALLET-01` opens `EXT-USER-01`'s profile and `EXT-WALLET-02` opens `EXT-USER-02`'s profile. Confirm that the user is granted access to the authenticated features. Repeat the process and reject the signature request; confirm that the user remains signed out and cannot access authenticated features.

2. ST-AUTH-EXT-2

   Control: Manual

   Initial State: The application is running on its public page in a supported browser. `EXT-WALLET-03` is available in a compatible browser extension and is not yet associated with a DAIIM user profile. The wallet owner is not signed in.

   Input:

   - Connect `EXT-WALLET-03` and approve the verification-message signature.
   - Sign out of the resulting account.
   - Reconnect `EXT-WALLET-03` and approve the verification-message signature again.

   Output:

- After the first approved signature, the user is authenticated and a profile is associated with `EXT-WALLET-03`. The system grants the user access to authenticated features.
- Sign-out ends the authenticated session; the user cannot access the profile or authenticated features while signed out.
- After the second approved signature, the same profile associated with `EXT-WALLET-03` is displayed. No second profile is created for the wallet. The system grants the user access to authenticated features.

Test Case Derivation: FR1 requires the system to verify ownership of the wallet address and associate that verified address with the same user profile on every successful authentication, including after sign-out. The second successful authentication must restore the profile created during the first sign-in.

How test will be performed: Record the number of DAIIM profiles associated with `EXT-WALLET-03`. Connect the wallet and approve the verification-message signature. Record the displayed profile identifier and confirm that the wallet is associated with that profile. Sign out and verify that the profile and authenticated features are no longer accessible. Sign in again with the same wallet and approve the signature. Verify that the displayed profile identifier matches the recorded identifier and that the number of profiles associated with the wallet has not increased.

**Authentication Using an Embedded Wallet**

1. ST-AUTH-EMB-1

   Control: Automatic

   Initial State: The application is running in a test environment with the embedded-wallet provider mocked. Identities `new.email@example.test`, `+15555550101`, and social-account subject `social-new-01` have no associated DAIIM account or embedded wallet. Configure the provider mock to return successful authentication and wallet creation for these identities, with distinct addresses `EMB-WALLET-NEW-01`, `EMB-WALLET-NEW-02`,

and `EMB-WALLET-NEW-03`, respectively. The malformed email `not-an-email` should be rejected by the system before the provider is called.

Input:

- Submit `new.email@example.test` using the email sign-in method.
- Submit `+15555550101` using the phone sign-in method.
- Submit social-account subject `social-new-01` using the supported social sign-in method.
- Submit malformed email `not-an-email` using the email sign-in method.

Output:

- For each of the three supported identities, the system creates one embedded wallet associated with the user's account, displays its corresponding address and a creation confirmation, authenticates the user, and makes the associated profile available.
- For `not-an-email`, the system reports an input error, displays no wallet-creation confirmation or address, and does not authenticate the user or create a wallet.

Test Case Derivation: FR2 requires successful creation for supported identity input after authentication, with the public address and confirmation as outputs. The system must reject malformed identity input before wallet creation. FR1 requires the successful wallet authentication to identify the user.

How test will be performed: For each valid test identity, call the corresponding embedded-wallet sign-in method with the specified email address, phone number, or social-account identity. For each successful case, compare the displayed address with the configured mock address, verify that the corresponding profile is authenticated, and confirm that exactly one wallet was created. For the malformed email, enter it using the email sign-in method and verify that the system reports an input error, does not call the provider, and does not create a wallet or authenticated session.

2. ST-AUTH-EMB-2

Control: Automatic

Initial State: The application is running in a test environment, and existing accounts `EMB-RETURN-01`, `EMB-RETURN-02`, and `EMB-RETURN-03` are signed out. Their embedded wallet addresses are respectively `EMB-WALLET-RETURN-01`, `EMB-WALLET-RETURN-02`, and `EMB-WALLET-RETURN-03`. Configure the embedded-wallet provider mock to return successful authentication responses with those address mappings. Configure it to return an authentication failure for `locked.user@example.test`, which has no associated account.

Input:

- Enter `returning.email@example.test` using the email sign-in method.
- Enter `+15555550102` using the phone sign-in method.
- Provide social-account subject `social-return-03` using the supported social sign-in method.
- Enter `locked.user@example.test` using the email sign-in method.

Output:

- For `returning.email@example.test`, the system authenticates `EMB-RETURN-01` and displays its existing profile and `EMB-WALLET-RETURN-01`.
- For `+15555550102`, the system authenticates `EMB-RETURN-02` and displays its existing profile and `EMB-WALLET-RETURN-02`.
- For social-account subject `social-return-03`, the system authenticates `EMB-RETURN-03` and displays its existing profile and `EMB-WALLET-RETURN-03`.
- For `locked.user@example.test`, the system reports authentication failure and does not create an authenticated session or grant access to a profile.

Test Case Derivation: FR1 requires each verified wallet address to remain associated with the same user profile after sign-out. Each successful response must restore the mapped existing profile rather than create another account. The failed authentication response does not verify wallet ownership, so it must not grant access.

How test will be performed: Call the application's embedded-wallet authentication entry point with the sign-in method and parameters from

each returning-user credentials. Configure the provider mock to return that fixture's existing wallet authentication response. Assert that each call returns an authenticated session for the user profile. Then call the same entry point with the sign-in method and identity parameters from the authentication-failure credentials. Assert that the call returns no authenticated session, grants no profile access and displays the appropriate error message.

### 4.1.2 Inference Task Creation

This test verifies the end-to-end task creation flow, including validation of the task definition, user authorization, and confirmation that the task-creation program instruction succeeds on the blockchain.

**Create and Validate Inference Tasks**

1. ST-TASK-CREATE-1

   Control: Automatic

   Initial State: The system is running on Solana Devnet. The requester wallet address fixture is authenticated, and the test harness is configured with its wallet signer and sufficient test SOL to pay transaction fees. The unique task names `Regression-Test-Task`, `Regression-Test-Min-Providers`, `Regression-Test-Invalid-Schema`, and `Rejected-Authorization-Task` are not already in use.

   Input:

   - A valid numerical regression task definition named `Regression-Test-Task`, including a description, numerical input and output schemas, an evaluation metric, a minimum provider count of 2 or more, reputation tolerance, acceptance period, and maximum prediction time.
   - The valid definition named `Regression-Test-Min-Providers`, with its minimum provider count changed to 1.
   - The valid definition named `Regression-Test-Invalid-Schema`, with an input or output schema containing non-numerical values incompatible with a numerical regression task.

- The valid definition named `Rejected-Authorization-Task`.

Output:

- For the valid definition with authorization approved, the task is created with the submitted definition, and the corresponding Solana Devnet transaction succeeds with the task-creation instruction.
- For the definition with fewer than 2 minimum providers, the system rejects the definition, and no task is created.
- For the definition with a schema incompatible with numerical regression, the system rejects the definition, and no task is created.
- For the valid definition with authorization rejected, no task is created and no successful task-creation transaction occurs.

Test Case Derivation: FR6 requires a task name, description, type, input schema, and output schema; the incompatible-schema case checks that the definition matches the numerical regression task. FR8 requires at least 2 providers, so a value of 1 must be rejected. FR5 permits authenticated users to create tasks, while FR4 requires wallet approval before an on-chain state change. Therefore, the valid task should be created only after approval, and rejecting authorization should leave it uncreated. FR50 and FR51 require a final transaction status and a transaction identifier that can be used to verify the successful instruction.

How test will be performed: Submit the valid definition to the task-creation flow and assert that the confirmation details match the submitted definition. Configure the test wallet signer to approve the authorization request. After confirmation, query Solana Devnet using the returned transaction identifier and assert that the transaction succeeded and contains the task-creation instruction. Query the application to confirm that `Regression-Test-Task` is available with the submitted definition. Submit the minimum-provider and invalid-schema definitions; assert that each is rejected before authorization and no task is created. Finally, submit the valid definition named `Rejected-Authorization-Task`, configure the signer to reject authorization, and assert that no task-creation instruction is submitted and no task is created.

### 4.1.3 Provider Registration and Staking

These tests verify the provider registration and staking requirements in the SRS. They cover registering as an inference provider for a task, locking the required minimum stake in escrow, unregistering when eligible, and viewing provider information through the dashboard.

#### Registering as an Inference Provider

1. ST-PROV-1

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for inference task `TASK-01`. The task requires a minimum stake of tokens, and the provider's wallet contains sufficient tokens and SOL to complete the transaction.

   Input:

   - Navigate to `TASK-01` and select the option to register as an inference provider.
   - Review the required minimum stake of tokens and the associated registration details.
   - Confirm the registration and approve the wallet transaction.

   Output:

   - The provider is successfully registered for `TASK-01`.
   - Exactly the minimum amount of tokens are locked in escrow as the provider's stake.
   - The provider is added to the task's list of registered providers and receives the base reputation score for the task.
   - The system displays the transaction status and, once confirmed, the transaction identifier.

   Test Case Derivation: FR_InferenceTaskRegistration requires authenticated users to be able to register as inference providers by staking the

required minimum amount of tokens. FR_InferenceTaskRegistrationStaking requires the minimum stake to be locked in escrow. The transaction authorization and confirmation behaviour is also covered by the related transaction requirements.

How test will be performed: Authenticate a provider account and navigate to `TASK-01`. Select the registration option and verify that the system displays the required minimum stake. Confirm the registration and approve the transaction through the connected wallet. After the transaction is confirmed, inspect the task's registered-provider list and the provider's account information. Verify that the provider is registered, has received the base reputation score, and that exactly the minimum amount of tokens have been transferred into escrow.

2. ST-PROV-2

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for `TASK-02`. The task requires a minimum stake of tokens, but the provider's wallet contains fewer than 100 tokens.

   Input:

   - Navigate to `TASK-02` and select the option to register as an inference provider.
   - Attempt to proceed with registration despite having an insufficient token balance.

   Output:

   - The system prevents the provider from completing registration.
   - An error message indicates that the provider's token balance is insufficient and identifies the required shortfall.
   - No stake is transferred into escrow, and the provider is not added to the task's list of registered providers.

Test Case Derivation: FR_InferenceTaskRegistration and FR_InferenceTaskRegistrationStaki require the provider to stake the required minimum amount of tokens to register. The related balance-check assumptions require the system to verify that sufficient funds are available before processing the transaction.

How test will be performed: Authenticate a provider account with fewer than 100 tokens and navigate to `TASK-02`. Attempt to register for the task. Verify that the system displays an insufficient-balance error, that no transaction transfers tokens into escrow, and that the provider does not appear in the task's registered-provider list.

**Unregistering from an Inference Task**

1. ST-PROV-3

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-03`. The provider has no active accepted inference requests for this task. The provider's current reputation score is at or above the task's reduced-stake-return threshold, and the provider has a remaining stake of tokens.

   Input:

   - Navigate to the provider's registration details for `TASK-03`.
   - Select the option to unregister from the task.
   - Review and confirm the unregistration.
   - Approve the wallet transaction, if prompted.

   Output:

   - The provider is successfully unregistered from `TASK-03`.
   - The provider is removed from the task's list of registered providers.
   - The provider's entire remaining stake of tokens is returned to their token account.

- The system displays the transaction status and, once confirmed, the transaction identifier.

Test Case Derivation: FR_ProviderWillingUnregistration requires the system to allow providers to unregister when they have no active accepted inference requests. FR_ProviderUnregistrationStakeReturn requires the entire remaining stake to be returned when the provider's reputation is at or above the task's reduced-stake-return threshold.

How test will be performed: Authenticate the registered provider and verify that they have no active accepted inference requests for `TASK-03`. Select the unregistration option and confirm the action. After the transaction is confirmed, inspect the provider's registration status, the task's registered-provider list, and the provider's token balance. Verify that the provider has been removed from the list and that the entire remaining stake has been returned.

2. ST-PROV-4

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-04`. The provider has accepted an inference request that is still active for this task.

   Input:

   - Navigate to the provider's registration details for `TASK-04`.
   - Select the option to unregister from the task.
   - Attempt to confirm the unregistration.

   Output:

   - The system rejects the unregistration attempt because the provider has an active accepted inference request.
   - An error message explains why the provider cannot unregister.
   - The provider remains registered for `TASK-04`, and their remaining stake is unchanged.

Test Case Derivation: FR_ProviderWillingUnregistration requires that a provider must not have accepted any active inference requests for the task before they can unregister.

How test will be performed: Authenticate a provider account that is registered for `TASK-04` and has an active accepted inference request. Attempt to unregister from the task. Verify that the system rejects the attempt, displays an appropriate error message, maintains the provider's registration, and does not modify the provider's remaining stake.

### Viewing the Provider Dashboard

1. ST-PROV-5

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for two inference tasks, `TASK-05` and `TASK-06`. The provider has an existing reputation score, staked token balance, and earned rewards recorded for each task.

   Input:

   - Navigate to the provider dashboard.
   - View the provider's information for `TASK-05` and `TASK-06`.

   Output:

   - The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-05`.
   - The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-06`.
   - The displayed values correspond to the provider's recorded information for each task.

   Test Case Derivation: FR_ProviderDashboard requires inference providers to be able to view their reputation score, staked tokens, and rewards earned for each inference task they are registered for.

   How test will be performed: Authenticate the provider account and navigate to the provider dashboard. Retrieve the provider's recorded

reputation, stake, and reward values for both tasks from the system's records. Compare these values with the information displayed on the dashboard and verify that the correct information is displayed for each task.

### 4.1.4 Inference Requests

These tests verify the inference request requirements in the SRS. They cover submitting requests for existing inference tasks, validating request input data against the task's input schema, authorizing payments into escrow, and broadcasting submitted requests to registered providers.

**Submitting an Inference Request**

1. ST-REQ-1

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-01`, is available and has at least two registered providers. The task's input schema requires a numerical input field named `input_value`. The requester's wallet contains sufficient tokens and SOL to submit the request.

   Input:

   - Navigate to `TASK-01` and select the option to submit an inference request.
   - Provide valid input data containing `input_value` with a numerical value
   - Review the request details and confirm the submission.
   - Authorize the payment through the connected wallet.

   Output:

   - The system accepts the input data and successfully submits the inference request for `TASK-01`.

- The request is assigned a unique identifier and its submission status is displayed to the requester.
- The input data is stored securely for processing and is not recorded directly on-chain.
- The requester is informed that the request has been submitted and is awaiting provider participation.

Test Case Derivation: FR_RequestSubmission requires authenticated users to be able to submit inference requests for existing tasks, with input data conforming to the task's input schema. The related input-data requirements specify that input data must be stored securely and not placed directly on-chain.

How test will be performed: Authenticate a requester account and navigate to `TASK-01`. Enter the valid input data and submit the request. Authorize the transaction through the connected wallet. Verify that the system creates a request with a unique identifier, displays the appropriate submission status, and securely stores the input data. Inspect the on-chain request record to confirm that the input data itself has not been recorded on-chain.

2. ST-REQ-2

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-02`, is available and has at least two registered providers. The task's input schema requires a numerical field named `input_value`.

   Input:

   - Navigate to `TASK-02` and select the option to submit an inference request.
   - Provide input data with the required `input_value` field missing.
   - Repeat the submission attempt with `input_value` provided as a non-numerical value.

   Output:

- The system rejects both invalid input submissions.
- The system displays an error message identifying the missing or invalid input field.
- No inference request is submitted, no request identifier is created, and no payment is transferred into escrow.

Test Case Derivation: FR_RequestValidation requires the system to reject inference requests whose input data does not conform to the task's input schema. FR_RequestSubmission requires submitted requests to include input data that conforms to that schema.

How test will be performed: Authenticate a requester account and navigate to `TASK-02`. Attempt to submit a request with the required input field missing. Verify that the system rejects the request and identifies the missing field. Repeat the test using a non-numerical value for `input_value`. Verify that the request is rejected again and that neither attempt creates a request or transfers funds into escrow.

**Request Payment and Authorization**

1. ST-REQ-3

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-03`, is available and has at least two registered providers. The task has a minimum payment requirement. The requester's wallet contains sufficient tokens and SOL to authorize a payment

   Input:

   - Navigate to `TASK-03` and provide valid input data.
   - Select a payment amount of tokens.
   - Review the transaction details and confirm the request submission.
   - Approve the transaction through the connected wallet.

   Output:

- The system accepts the payment amount because it exceeds the task's minimum payment requirement.
- The wallet prompts the requester to authorize a transaction transferring the tokens into escrow.
- After authorization and transaction confirmation, exactly those amount of tokens are transferred into escrow.
- The system displays the transaction status and, once confirmed, the transaction identifier.
- The amount transferred into escrow matches the amount authorized by the requester.

Test Case Derivation: FR_RequestPayment requires the requester to choose a token payment greater than the minimum and authorize a wallet transaction transferring that amount into escrow. The related transaction authorization and preview requirements ensure that the requester can review and authorize the transaction. SA_ChargeEqualsAuthorized requires the amount charged to match the amount authorized.

How test will be performed: Authenticate a requester account and navigate to `TASK-03`. Enter valid request input and select a payment amount for a set amount of tokens. Review the transaction preview and approve the transaction through the connected wallet. After confirmation, inspect the requester's token balance and the escrow account's balance. Verify that exactly the set amount of tokens have been transferred into escrow and that the amount matches the requester's authorization.

**Broadcasting Submitted Inference Requests**

1. ST-REQ-4

   Control: Automatic

   Initial State: The system is running in a test environment. An existing inference task, `TASK-04`, has three registered providers: `PROV-01`, `PROV-02`, and `PROV-03`. All three providers are available to receive inference requests. The task has a configured acceptance period and maximum prediction time.

Input:

- Submit a valid inference request for `TASK-04`.
- Set the proposed payment amount to a number of tokens.
- Configure the acceptance-period expiry time to 10 minutes after submission.
- Configure the maximum prediction time to 30 minutes.

Output:

- The system successfully submits the inference request.
- The system broadcasts the request to all three registered providers.
- Each provider receives the task definition, proposed payment amount, acceptance-period expiry time, and maximum prediction time.
- The broadcast information is consistent across all three providers and matches the submitted request.

Test Case Derivation: FR_RequestBroadcast requires the system to broadcast the task definition, proposed payment amount, acceptance-period expiry time, and maximum prediction time to all providers registered for the task after an inference request is submitted.

How test will be performed: Configure `TASK-04` with the three specified registered providers and submit a valid inference request with the specified payment amount and timing values. Monitor the messages or notifications received by each provider. Verify that all three providers receive the broadcast and that the task definition, proposed payment, acceptance-period expiry time, and maximum prediction time match the submitted request.

## 4.2 Tests for Nonfunctional Requirements

### 4.2.1 Area of Testing1

**Title for Test**

1. test-id1

Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

#### 4.2.2 Area of Testing2

...

### 4.3 Traceability Between Test Cases and Requirements

## 5 Unit Test Description

### 5.1 Unit Testing Scope

### 5.2 Tests for Functional Requirements

#### 5.2.1 Module 1

1. test-id1

   Type:

   Initial State:

   Input:

   Output:

Test Case Derivation:

How test will be performed:

2. test-id2

   Type:

   Initial State:

   Input:

   Output:

   Test Case Derivation:

   How test will be performed:

3. ...

#### 5.2.2 Module 2

...

### 5.3 Tests for Nonfunctional Requirements

#### 5.3.1 Module ?

1. test-id1

   Type:

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

Input:

Output:

How test will be performed:

### 5.3.2 Module ?

...

## 5.4 Traceability Between Test Cases and Modules

# 6 Appendix

This is where you can place additional information.

## 6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

## 6.2 Usability Survey Questions?

# 7 Appendix — Generative AI Use Disclosure

## 7.1 AI Tools Used

An AI assistant using the Copilot SDK in Visual Studio Code was used to assist with revisions to this document. The specific model used is GPT-6 Luna.

## 7.2 How and Where Used

The AI assistant was used to discuss, draft, and revise prose and requirements for the verification and validation work. In the VnV Plan, it helped structure system tests by functional area and draft and refine test cases for wallet authentication, embedded-wallet creation, returning-user authentication, and inference-task creation. Revisions focused on making inputs, expected outputs, test procedures, and requirement derivations clearer and more specific.

The assistant also helped revise related SRS content: clarifying support for embedded and browser-extension wallets, adding requirements for embedded-wallet creation and persistent wallet-to-profile association, and adding a conservative User Profile entry to the major data entities and data dictionary. The team directed the scope and reviewed or adjusted the proposed content.

## 7.3 Verification of AI-Generated Content

AI-assisted content in this document was checked against the SRS to confirm that each test case addresses stated requirements and does not introduce unapproved system behaviour or design decisions. Requirement references and the expected outcomes derived from them were checked against the corresponding SRS requirements and use cases. Test cases were reviewed for clear inputs, expected results, and procedures, and the LaTeX structure and cross-references were inspected after editing. Test fixtures, schema examples, and blockchain verification details are proposed examples and must be reconciled with the implemented system before execution. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

# 8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?